

ETL + Feature Engineering com Apache Airflow

IMD3005 - MLOPS

Prof. Adelson de Araújo

Neste laboratório, você irá construir e executar um pipeline de ETL para Feature Engineering, utilizando o Apache Airflow para definir e gerenciar o workflow.

Ao final da atividade, você deverá ser capaz de:

- obter dados de uma fonte externa por meio de uma API;
- realizar transformações e/ou extração de features;
- organizar essas etapas como um DAG do Airflow;
- executar o workflow pela interface do Airflow;
- gerar um conjunto de dados contendo as features produzidas;
- compreender as vantagens de automatizar esse processo por meio de um workflow.

O problema

Imagine que você está preparando dados de uma API pública para uso futuro em um sistema de Machine Learning, mas os dados ainda não estão em um formato adequado. Seu trabalho será construir um pipeline que extraia os dados, aplique feature engineering e salve o resultado em um arquivo CSV, implementado como um DAG do Apache Airflow. O objetivo não é construir um modelo de Machine Learning, e sim automatizar a preparação dos dados que poderiam posteriormente ser utilizados por um modelo.

Etapa 0 — Instalação do Airflow

Antes de iniciar, você precisa ter o Apache Airflow instalado e rodando localmente.

Linux / macOS

Crie um ambiente Python dedicado e instale o Airflow.

```
conda create -n airflow-env python=3.11
conda activate airflow-env
pip install "apache-airflow" # veja o comando completo no guia oficial
airflow standalone
```

O comando de instalação via pip varia conforme a versão do Python e do Airflow (usa constraints). Siga o guia oficial: github.com/apache/airflow/blob/main/INSTALLING.md

O comando `airflow standalone` inicia o servidor web, o scheduler e cria automaticamente um usuário administrador. As credenciais de acesso ficam salvas em:

```
~/airflow/simple_auth_manager_passwords.json.generated
```

Para visualizá-las:

```
cat ~/airflow/simple_auth_manager_passwords.json.generated
```

Acesse a interface em <http://localhost:8080> utilizando essas credenciais.

Windows (via WSL)

O Airflow não é oficialmente suportado no Windows nativo. Utilize o WSL (Windows Subsystem for Linux) para rodar um ambiente Linux dentro do Windows:

1. abra o PowerShell como administrador e instale o WSL com `wsl --install`; reinicie o computador quando solicitado;
2. na primeira execução, defina um nome de usuário e senha para o Ubuntu;
3. abra o terminal Ubuntu (WSL) e siga os mesmos passos da seção Linux acima para instalar o Miniconda e o Airflow;
4. como o WSL2 encaminha portas automaticamente, acesse a interface do Airflow normalmente pelo navegador do Windows em <http://localhost:8080>.

IMPORTANTE

Se você ainda não tiver o Miniconda/Anaconda instalado no WSL, baixe-o em docs.conda.io antes de criar o ambiente.

Etapa 1 — Escolha da fonte de dados

Escolha uma API pública que disponibilize dados que possam ser processados com Python. Algumas possibilidades:

- dados meteorológicos, de países e cidades, filmes ou livros;
- dados esportivos, financeiros, de transporte ou ambientais;
- dados de notícias, ou outra API de sua escolha.

Requisitos

- permitir acesso sem autenticação ou utilizar uma chave gratuita;
- retornar dados estruturados, preferencialmente em JSON;
- disponibilizar dados suficientes para a criação de novas features;
- permitir que o pipeline seja executado novamente.

DICA

Escolha uma API simples. O objetivo principal do laboratório é praticar ETL + Feature Engineering + Airflow, e não desenvolver uma integração complexa com uma API.

Etapa 2 — Extração

Crie uma função de extração, responsável por extrair os dados da API, a qual deve:

5. realizar uma requisição para a API e verificar se ela foi bem-sucedida;
6. obter os dados retornados e organizá-los em uma estrutura adequada para a etapa posterior.

Etapa 3 — Feature Engineering

Crie uma segunda função responsável por transformar os dados extraídos em um conjunto de features, utilizando pelo menos duas técnicas apresentadas durante as aulas/notebook. Alguns exemplos, conforme a modalidade dos dados:

Categoria	Exemplos
Transformação	normalização, padronização, transformação de distribuição, codificação de variáveis categóricas
Extração	features temporais, lags e decomposição de séries temporais, TF-IDF, embeddings, features espaciais, entre outras vistas em aula
Seleção	técnicas de seleção de features, quando adequadas aos dados escolhidos

Etapa 4 — Persistência e montagem do DAG

Crie uma terceira função responsável por persistir o resultado do Feature Engineering em features.csv.

As três funções devem compor um único DAG, com dependências que representem o fluxo de dados:



O código deve deixar clara a separação entre as responsabilidades de cada tarefa:

```
def extract_task():
    # 1. Obtém dados da API
    ...

def transform_task():
    # 2. Realiza Feature Engineering
    ...

def load_task():
    # 3. Persiste o resultado em features.csv
    ...
```

Etapa 5 — Execução e exploração no Airflow

1. coloque o arquivo .py na pasta dags/ e aguarde o Airflow identificar o DAG;
2. acesse a interface web, localize e habilite o DAG;
3. execute-o manualmente e acompanhe a execução das tarefas;
4. verifique se todas as tarefas concluíram com sucesso e se features.csv foi gerado.

Depois de executar, explore também na interface:

- as dependências entre as tarefas e o estado/tempo de execução de cada uma;
- eventuais erros e os logs das tarefas.

Use o módulo de Variáveis no Airflow

Utilize Airflow Variables para armazenar parâmetros de configuração do pipeline (por exemplo: API_URL, OUTPUT_PATH, CITY, N_RECORDS). Esses valores não devem precisar ser alterados diretamente no código da tarefa. **Importante:** não é necessário utilizar Variables para informações que não sejam realmente configurações. Use-as quando houver um parâmetro que faça sentido alterar sem modificar o código do DAG.

Organização do projeto e do DAG

Uma organização mínima do projeto, tendo o DAG como arquivo principal:

```
laboratorio1/  
├── dags/  
│   └── dag_feature_engineering.py  
├── data/  
│   └── features.csv  
└── README.md
```

IMPORTANTE

Não é necessário implementar um sistema complexo. O foco é demonstrar que você compreende como decompor um pipeline de dados em tarefas gerenciáveis por um workflow.

Um bom pipeline deve poder ser executado novamente sem exigir que o programador apague manualmente arquivos ou altere o código. Teste seu DAG executando-o mais de uma vez e verifique:

- o pipeline funciona novamente?
- o arquivo de saída é sobrescrito ou atualizado adequadamente?
- o resultado continua consistente?
- alguma etapa depende indevidamente de uma execução anterior?

Entrega

A entrega deverá ser realizada pelo SIGAA. Envie um único arquivo laboratorio1_NOME.zip, contendo:

```
laboratorio1_NOME/  
├── dag_feature_engineering.py  
├── features.csv  
└── relatorio.pdf
```

Relatório

O relatório do laboratório 1 deverá apresentar brevemente:

1. Dados

- Qual API foi utilizada?
- Que tipo de dados ela fornece?
- Por que essa fonte foi escolhida?

2. Feature Engineering

- Quais features foram produzidas e com quais técnicas?
- Por que essas técnicas são adequadas aos dados?

4. Pipeline no Airflow

Inclua uma captura de tela da interface mostrando o DAG executado com sucesso, e explique brevemente:

- Apresente o fluxo implementado (API → Extract → Transform → Load → CSV) e explique brevemente a responsabilidade de cada tarefa.
- Como o DAG foi executado e como você verificou o resultado? Quais informações dos logs foram úteis?
- Quais são os principais benefícios do Apache Airflow para o gerenciamento de pipelines de Machine Learning? Considere aspectos como automação, dependências entre tarefas, agendamento, monitoramento, logs, reexecução e escalabilidade.
- Instalação: Quando seria mais apropriado utilizar um fluxo ETL e quando seria mais apropriado utilizar ELT? Relacione sua resposta com onde ocorre a transformação, capacidade de processamento, armazenamento dos dados, flexibilidade do pipeline e contexto de utilização.
- ETL × ELT: Quais foram as principais dificuldades encontradas para instalar e executar o Airflow? Como você resolveu esses problemas?

Checklist antes da entrega

Item	✓
Minha API está funcionando.	
O DAG possui pelo menos 3 tarefas (Extração, Feature Engineering, Persistência).	
As dependências entre as tarefas estão corretas.	
Utilizei pelo menos duas técnicas de Feature Engineering e justifiquei as escolhas.	
O DAG foi executado pela interface do Airflow e todas as tarefas terminaram com sucesso.	
O arquivo features.csv foi produzido.	
Testei executar o DAG novamente.	
Incluí uma captura de tela da execução.	
Respondi às três questões do relatório.	
O ZIP contém todos os arquivos solicitados.	